



Cairo University
Faculty of Engineering
Electronics & Communications Dept.



NN ASSIGNMENT 2

Artificial Neural Networks

Technical Report

Prepared by:

Abdallah Essam Mohamed Mahmoud

ID: ...

Zeyad Antar Othman Abu-Zaid

ID: 9220331

Abdelrahman Hatem Nasr Youssef

ID: 9220461

Submitted to:

Prof. Dr. Mohsen Rashwan

Dr. Mohamed Abdelghani

April 2026

Contents

1	Problem 1: Training Feed-Forward Neural Networks (MLP)	2
1.1	Introduction	2
1.2	Results	2
1.3	Analysis and Discussion	3
2	Problem 4: Speech Digit Recognition using Autoencoders	5
2.1	Introduction	5
2.2	Baseline: Summary Features + SVM	5
2.3	Autoencoder Approach	5
2.4	Results	6
2.5	Analysis and Discussion	7

1 Problem 1: Training Feed-Forward Neural Networks (MLP)

1.1 Introduction

This part explores the classification of the **ReducedMNIST** handwritten digit dataset using **Multilayer Perceptrons (MLPs)**. The objective is to evaluate how *network depth* influences performance when combined with different input feature representations.

A Multilayer Perceptron is a fully connected feed-forward neural network. Every neuron in one layer connects to every neuron in the next layer. It is the simplest form of “deep” learning:

- **Input layer:** receives the feature vector (e.g. 225 DCT coefficients).
- **Hidden layers:** apply a learned linear transformation followed by a non-linear activation function (ReLU in our case). More hidden layers = deeper network = ability to learn more complex patterns.
- **Output layer:** 10 neurons (one per digit 0–9), producing class probabilities via the Softmax function.

We trained MLPs with **1, 3, and 4 hidden layers** using features derived from:

1. **Discrete Cosine Transform (DCT)** — 225 features (top-left 15×15 block).
2. **Principal Component Analysis (PCA)** — 262 components retaining 95% of variance.
3. **Autoencoder (AE)** — a neural network trained to compress images into 64 features.

The dataset is a reduced version of the standard MNIST with 10,000 training images (1,000 per digit) and 2,000 test images (200 per digit), each 28×28 pixels normalised to $[0, 1]$.

1.2 Results

System: Intel Core i7-9850H @ 2.60 GHz, 32 GB RAM, Windows 11, Python 3.12, PyTorch 2.11 (CPU only).

Hyper-parameters: Epochs = 10, Batch size = 64, Learning rate = 10^{-3} , Optimizer = Adam, Activation = ReLU, Loss = CrossEntropyLoss. Autoencoder pre-training: 20 epochs, bottleneck = 64, MSE loss.

Hidden Layers	DCT		PCA		AutoEncoder	
	Acc (%)	Time (ms)	Acc (%)	Time (ms)	Acc (%)	Time (ms)
1 layer [128]	95.8	3758.3	93.5	4470.1	96.4	3944.1
3 layers [256-128-64]	96.4	6096.5	93.8	5808.2	97.2	6135.4
4 layers [256-128-64-32]	95.5	7154.5	93.3	7322.9	96.7	6417.3

Table 1. Test accuracy and processing time for each feature–depth combination.

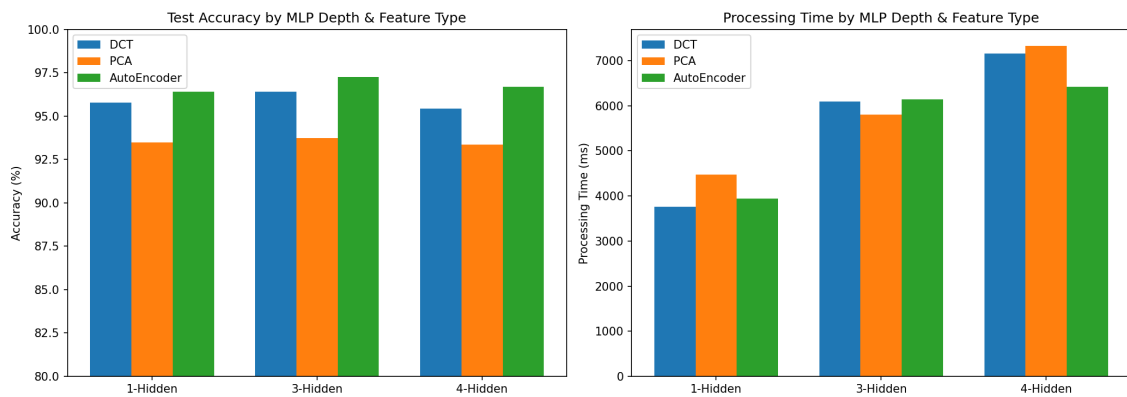


Figure 1. Comparison of test accuracy (left) and processing time (right) across all feature types and MLP depths.

1.3 Analysis and Discussion

1.3.1 Effect of Network Depth and Overfitting

Increasing depth from 1 to 3 hidden layers gave a modest accuracy boost for DCT (95.8% → 96.4%) and AutoEncoder (96.4% → 97.2%), but going to 4 hidden layers *decreased* accuracy in every feature type — DCT dropped to 95.5%, PCA to 93.3%, and AutoEncoder to 96.7%.

This accuracy drop at 4 layers is a clear sign of **overfitting**:

- **Too many parameters for the data:** the 4-layer MLP (256–128–64–32) has significantly more trainable weights than the 1-layer variant. With only 1,000 training samples per class, the extra parameters fit noise in the training set rather than learning generalisable patterns, leading to lower test accuracy.
- **Vanishing gradient effect:** deeper networks propagate smaller gradients to early layers, making it harder for those layers to learn useful features in only 10 epochs.
- **No regularisation:** the models use no dropout, weight decay, or early stopping, so deeper architectures are especially prone to memorising the training data.

- **Sweet spot at 3 layers:** the 3-layer MLP strikes the best balance — enough capacity to capture non-linear patterns without overfitting the limited dataset.

1.3.2 Effect of Feature Type

The **Autoencoder features achieved the highest accuracy** (97.2%) despite being the smallest representation (only 64 dimensions).

Ranking: AutoEncoder (97.2%) > DCT (96.4%) > PCA (93.8%)

Why AutoEncoder outperforms:

- The autoencoder learns a **non-linear** mapping, allowing it to capture complex relationships between pixels that linear PCA cannot.
- Its 64-D features are extremely compact yet highly discriminative — the bottleneck forces it to retain only the most class-relevant information.
- The autoencoder is trained *on the same data*, so its features are tailored to this specific dataset.

Why DCT outperforms PCA:

- DCT captures frequency information which is well-suited for digit recognition (global shape patterns).
- PCA with 262 components retains some noisy dimensions, which can slightly confuse the classifier.

1.3.3 Processing Time

Processing time scales with both **network depth** and **input dimensionality**:

- PCA (262-D input) consistently took the longest due to the larger first weight matrix.
- AutoEncoder (64-D input) was the fastest despite achieving the best accuracy — demonstrating that good feature engineering reduces both compute cost and error rate.
- Going from 1 to 4 hidden layers roughly doubles the processing time (e.g. DCT: 3758 → 7155 ms), while accuracy does not improve proportionally.

2 Problem 4: Speech Digit Recognition using Autoencoders

2.1 Introduction

This problem explores **speech digit recognition** (digits 0–10) using audio recordings. The goal is twofold: (1) build a **baseline** classifier from hand-crafted audio features, and (2) show that an **autoencoder** can learn a superior latent representation that significantly outperforms the baseline.

Key concepts:

- **Utterance:** one complete audio recording of a person saying a digit.
- **Frame:** a short 15 ms slice of the waveform (with 10 ms hop). Speech is approximately stationary over such short durations.
- **MFCC:** Mel-Frequency Cepstral Coefficients — 30 coefficients per frame that mimic how the human ear perceives pitch and timbre.
- **Delta & Delta-Delta:** first- and second-order time derivatives of MFCCs that capture velocity and acceleration of spectral change.

The dataset contains **1,200 training** and **300 testing** utterances (files named `{SpeakerID}_{digit}.wav`). All utterances have 81 frames after MFCC extraction, so the flattened input is $30 \times 81 = 2,430$ dimensions.

2.2 Baseline: Summary Features + SVM

The baseline compresses each utterance into a **180-D summary vector**:

$$\mathbf{x}_{\text{summary}} = [\mu(\text{MFCC}), \sigma(\text{MFCC}), \mu(\Delta), \sigma(\Delta), \mu(\Delta^2), \sigma(\Delta^2)] \in \mathbb{R}^{180}$$

where μ and σ denote the mean and standard deviation across all frames. This discards temporal ordering but retains global spectral statistics.

An **SVM** with RBF kernel is trained via `GridSearchCV` (5-fold CV) over $C \in \{1, 10, 100, 1000\}$ and $\gamma \in \{\text{scale}, 0.01, 0.001\}$.

2.3 Autoencoder Approach

2.3.1 Architecture

The supervised autoencoder processes the full flattened utterance ($\mathbb{R}^{2430}$) and consists of:

- **Encoder:** $2430 \rightarrow 1024 \rightarrow 512 \rightarrow 256$ (each with BatchNorm + ReLU)

- **Decoder:** $256 \rightarrow 512 \rightarrow 1024 \rightarrow 2430$
- **Classifier head:** Linear($256 \rightarrow 10$) branching from the latent layer

2.3.2 Training

The autoencoder is trained with a **joint loss**:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \lambda \cdot \mathcal{L}_{\text{cls}} \quad \text{where} \quad \mathcal{L}_{\text{recon}} = \text{MSE}(\mathbf{x}, \hat{\mathbf{x}}), \quad \mathcal{L}_{\text{cls}} = \text{CrossEntropy}(\mathbf{y}, \hat{\mathbf{y}})$$

with $\lambda = 2.0$ to emphasise classification.

Data augmentation: 3 noisy copies of each training sample (Gaussian noise, $\sigma = 0.05$) expand the training set from 1,200 to **4,800 samples**.

Training runs for **500 epochs** with Adam ($\text{lr} = 10^{-3}$), batch size 32. The classification loss $\mathcal{L}_{\text{cls}}$ is the cross-entropy loss of the classifier head attached to the latent layer; it drops to ≈ 0 by epoch 120, indicating perfect classification on training data. The reconstruction loss continues to decrease, reaching ≈ 0.073 by epoch 500.

2.3.3 Classification Strategies

Three strategies are evaluated on the test set:

- Classifier head:** direct output of the AE's built-in linear classifier.
- SVM on latent vectors:** train an SVM on the 256-D encoder output.
- SVM on hybrid features:** concatenate the scaled 256-D latent vector with the scaled 180-D summary features ($\mathbb{R}^{436}$) and train an SVM.

2.4 Results

System: same as Problem 1.

Method	Accuracy (%)	Train Time (s)	Test Time (ms)
Baseline (Summary + SVM)	70.00	10.5	60.2
AE Classifier Head	86.00		
AE Latent + SVM	89.33	2244.1	92.9
AE Hybrid + SVM (best)	90.67		

Table 2. Problem 4: Accuracy comparison of baseline vs. autoencoder-based strategies. AE training time includes 500 epochs of AE training plus SVM GridSearchCV.

Hyperparameter	Value
MFCCs	30 coefficients
Frame / Hop	15 ms / 10 ms
Latent dimension	256
AE epochs	500
Augmentation	3 noisy copies ($\sigma = 0.05$)
Loss weight (λ)	2.0
SVM kernel	RBF
Best SVM params	$C = 1, \gamma = \text{scale}$

Table 3. Problem 4: Key hyperparameters.

2.5 Analysis and Discussion

2.5.1 Why the Autoencoder Outperforms the Baseline

The baseline achieves **70.00%** by averaging MFCCs across time, which *discards all temporal information* — two utterances with identical average spectra but different time evolution become indistinguishable.

The autoencoder processes the **full frame sequence** (2,430-D input) and learns a 256-D latent code that captures both **spectral content** and **temporal dynamics**, yielding **90.67%** accuracy — a **+20.67 percentage point** improvement.

2.5.2 Hybrid Features

The best strategy combines the autoencoder latent vector with the summary features (each independently standardised). This works because:

- The latent vector captures temporal patterns the summary misses.
- The summary features capture global statistics (mean/std of deltas) that complement the AE's learned representation.
- Separate standardisation prevents one feature set from dominating due to scale differences.

2.5.3 Supervised AE Joint Loss

Adding the classification loss ($\lambda = 2.0$) to the reconstruction loss **shapes the latent space to be class-discriminative**, not just reconstruction-optimal. The classification loss drops to zero

on training data by epoch 120, while reconstruction loss continues to decrease — the model simultaneously learns to reconstruct and classify.

2.5.4 Data Augmentation

Gaussian noise augmentation (3×) expanded the training set from 1,200 to 4,800 samples, which:

- Reduces overfitting in the 500-epoch training regime.
- Teaches the AE to be robust to noise, improving generalisation.